



NavSight
Visual Odometry & Navigation

Roey Ben Harush, 315676163,
Tamir Sobuh, 206029084,
Morad Zubidat , 208156828

Supervised by:
Amit Dunskey

SRS SUBMITTED FOR THE DEGREE
"SOFTWARE ENGINEERING B.Sc."

Shenkar College
Faculty of Engineering

Supervisor: _____

Date: _____

Abstract

Modern navigation systems rely heavily on Global Navigation Satellite Systems (GNSS) such as GPS and Galileo. However, these systems suffer from significant limitations in dense urban environments, tunnels, or areas affected by interference and spoofing. As a result, navigation performance often degrades in conditions where satellite signals are unavailable or unreliable, limiting the effectiveness of GNSS-based positioning in complex cityscapes.

In this project, we are developing **NavSight** - an innovative **Visual Positioning and Dead Reckoning (VPS+DR)** system designed to enable real-time navigation and motion estimation without relying on GNSS signals. The system combines real-time image processing from the smartphone's camera with inertial motion data (IMU) to estimate position and orientation even when satellite signals are blocked or distorted.

NavSight employs **Visual-Inertial Odometry (VIO)** techniques to fuse visual and inertial data for continuous motion tracking. This approach allows the application to calculate the user's path in real time, apply drift compensation, and visualize movement directly on the device's screen.

The mobile application is being developed as a **native Android solution using Java or Kotlin** and OpenCV, featuring an intuitive, lightweight, and rider-oriented user interface specifically designed for **electric scooter navigation** in urban environments.

The goal of this project is to demonstrate the feasibility of a fully GNSS-independent visual navigation system and establish a foundation for future developments in urban mobility, intelligent transportation, and robotics.

Keywords: Computer Vision, Visual-Inertial Odometry, Dead Reckoning, Visual Navigation, GNSS-Denied Environments

Table of Contents

1.	Introduction	2
1.1.	Overview.....	2
1.2.	Problem Description and Motivation.....	2
1.3.	Goals.....	2
1.4.	Scope.....	2
1.5.	Glossary.....	2
2.	General description	2
2.1.	User Characteristics.....	2
2.1.1.	Stakeholders: Client Description.....	2
2.1.2.	End-Users Description and Scenarios.....	2
2.2.	The approach.....	2
2.3.	Constraints.....	2
2.4.	Assumptions and Dependencies.....	2
3.	Functional requirements	2
4.	Non-Functional requirements	2
5.	System flows	2
6.	Risk management	2
7.	System main screen specifications	2
8.	Non-goals	2
9.	Open issues	2
10.	References	2

1. Introduction

Positioning and navigation are essential components of modern life, supporting transportation, logistics, autonomous systems, and micromobility applications. Most existing solutions depend on Global Navigation Satellite Systems (GNSS), a collective term that includes GPS (United States), GLONASS (Russia), Galileo (EU), and BeiDou (China).

While GNSS provides high accuracy under open-sky conditions, it suffers from severe limitations in dense urban areas, tunnels, or environments affected by signal interference, jamming, or spoofing.

To overcome these challenges, this project introduces **NavSight**, a **Hybrid Visual Positioning System (VPS)** that combines **Visual-Inertial Odometry (VIO)** and **Dead Reckoning (DR)** to estimate the user's movement and orientation.

While visual odometry uses camera input to calculate motion between consecutive frames, dead reckoning leverages inertial sensor data (accelerometer and gyroscope) to maintain positional estimation when visual tracking becomes unreliable, such as during low lighting, motion blur, or visual occlusion.

This hybrid approach allows NavSight to sustain accurate navigation even when the camera temporarily loses track of visual landmarks.

By fusing these two complementary techniques, NavSight delivers a robust and reliable **GNSS-free localization solution** for electric scooter riders and other micromobility users in challenging urban environments. The system operates entirely offline, processes visual and inertial data in real time, and is implemented as a developed as a **native Android solution using Java or Kotlin** and OpenCV.

1.1. Overview

NavSight is a real-time mobile application that performs hybrid positioning using both visual and inertial data.

The system integrates **Visual-Inertial Odometry** for feature-based motion estimation and **Dead Reckoning** for continuous tracking when visual data is insufficient.

Through optimized computer vision algorithms and sensor fusion techniques, the app reconstructs the user's relative movement path on-screen.

The application is developed as a **native Android solution using Java or Kotlin** and leverages native C++ modules for performance-critical processing.

1.2. Problem Description and Motivation

GNSS signals are unreliable or unavailable in many everyday environments, such as tunnels, underground parking, or dense city centers.

These disruptions can cause severe positioning errors for micromobility users, leading to unsafe navigation or complete tracking loss.

NavSight addresses this gap by combining **vision-based** and **inertial-based** methods to maintain navigation continuity even in GNSS-denied conditions.

This hybrid approach provides resilience against temporary visual loss or environmental limitations, ensuring smoother and more reliable tracking.

1.3. Goals:

- Develop a **hybrid Visual-Inertial navigation system** combining VO and DR for real-time tracking.
- Maintain **continuous localization** even when the camera view is partially obstructed or motion-blurred.
- Support **offline operation**, independent of GNSS and internet connectivity.
- Achieve **real-time performance** on mid-range smartphones (<200ms latency).
- Demonstrate feasibility for **micromobility navigation** (e-scooters, bicycles, etc.).

1.4. Scope:

The NavSight project includes the design, implementation, and testing of a hybrid camera sensor-based positioning system for smartphones.

Key scope elements:

- Visual Odometry for motion estimation via OpenCV.
- Dead Reckoning using the smartphone's IMU (accelerometer + gyroscope).
- Fusion logic combines both inputs for robust tracking.
- Native Android (Java/Kotlin) mobile app with a live map interface..
- Field testing in diverse urban environments.

Out of scope:

- 3D environment reconstruction or SLAM mapping.
- Cloud-based data storage or external sensor hardware integration.

1.5. Glossary:

Term	Definition
VPS (Visual Positioning System)	System estimating position using visual data instead of GNSS.
VO (Visual Odometry)	Technique for estimating camera movement by analyzing sequential images.
DR (Dead Reckoning)	Estimating position by integrating velocity and direction from sensors (IMU).
VIO (Visual-Inertial Odometry)	Combining camera and IMU data for improved motion tracking accuracy.
IMU (Inertial Measurement Unit)	A sensor combining an accelerometer and gyroscope to measure motion.
GNSS	Global Navigation Satellite Systems (GPS, GLONASS, Galileo, BeiDou).
OpenCV	Open-source library for computer vision tasks.

2. General description

The *NavSight* project aims to develop a mobile navigation application capable of determining position and orientation without relying on GNSS signals. The system utilizes visual input from the smartphone's camera combined with inertial sensor data to estimate real-time movement and direction. This section outlines the system's users, the chosen technological approach, the constraints under which it operates, and the main assumptions and dependencies related to its development.

2.1. User Characteristic

2.1.1. Stakeholders: Client Description

The primary stakeholders are:

- **Shenkar Faculty of Engineering** - supervising and evaluating the project as part of the Software Engineering B.Sc. program.
- **Supervisors (Amit Dunskey)** - providing academic guidance and technical review.
- **Project Team (Roey Ben Harush, Tamir Sobuh, and Morad Zubidat)** - responsible for the system's design, development, testing, and documentation.
- **Potential Industry Stakeholders** - future adopters from the micromobility, robotics, or AR navigation industries.

2.1.2. End-Users Description and Scenarios

The target end-users are **electric scooter riders and pedestrians** navigating in dense urban environments where GNSS may be unreliable or unavailable.

Typical usage scenarios include:

- Navigating through **urban canyons** surrounded by tall buildings.
- Riding in **tunnels or underpasses** where GPS signals are blocked.
- Using **offline navigation** in areas without internet connectivity.
- Supporting **short-distance route tracking** with high visual accuracy.

Users are expected to have basic familiarity with smartphone applications, but do not require technical expertise. The interface is designed to be intuitive and minimal, with real-time visual feedback and clear route visualization.

Scenarios :

Title: User successfully tracks a route from start to finish.

Actor: Rider

Preconditions:

1. The NavSight application is installed on the smartphone.
2. The user has granted camera and sensor (motion) permissions.
3. The device is in an environment with sufficient visual features and lighting.

Main Success Scenario (Sunny Day):

1. User opens the NavSight application.
2. The system displays the main screen with a live camera feed.
3. The system prompts the user to perform initial calibration, user holds the device steady for 3-5 seconds.
4. The status panel indicates "Ready".
5. User presses the "Start" button.
6. The system begins tracking, and the status changes to "Tracking". A path starts to draw on the screen, following the user's movement.
7. The user navigates their route.
8. The user presses the "Stop" button upon completion.
9. The system stops tracking and displays a session summary with total distance and duration.

Alternative Flow 1: Degraded Visuals

1. At step 7, the user enters an area with low light or rapid motion, causing VIO to fail.
2. The system automatically switches to Dead Reckoning (DR) mode.
3. The status panel updates to "Tracking (DR Mode)".
4. The system continues to estimate the path using IMU data.
5. When visual quality improves, the system automatically switches back to VIO-fused tracking.

Alternative Flow 2: User Pauses Session

1. At step 7, the user presses the "Pause" button.
2. The system suspends tracking and timer. The status changes to "Paused".
3. The user presses the "Resume" button.
4. The system resumes the tracking session from where it left off.

2.2. The approach

NavSight's development approach combines:

1. **Visual-Inertial Odometry (VIO):** This is the core technical method. It **fuses** two data sources to precisely estimate motion and orientation:
 - **Visual Data:** Using OpenCV to detect and track features between camera frames.
 - **Inertial Data:** Using the IMU (accelerometer and gyroscope) data.
2. **Native Android Development:** Implemented using **Java or Kotlin in Android Studio** utilizing the Java Native Interface (JNI) to run the performance-critical VIO and OpenCV components.
3. **Real-Time, On-Device Processing:** All VIO algorithms are optimized to run entirely on the device, requiring no external servers or cloud processing for immediate results.
4. **Offline Operation:** Designed for full functionality in **GNSS-denied environments**, operating without any need for an internet connection or GPS signal.

This hybrid approach enables accurate, continuous position tracking even in GNSS-denied environments.

2.3. Constraints

NavSight operates under several technical and environmental constraints:

- **Hardware Limitations:** Performance depends on smartphone camera quality, sensor precision, and processing power.
- **Lighting Conditions:** Poor or rapidly changing light can affect visual tracking reliability.

- **Motion Blur:** Excessive scooter speed may reduce image clarity for feature detection.
- **Power Consumption:** Continuous camera and sensor use may increase battery drain.
- **No External Data:** The app must function completely offline without cloud support.

2.4. Assumptions and Dependencies

The system's development and performance rely on the following assumptions and dependencies:

- The device has a functional rear camera, accelerometer, and gyroscope.
- The environment contains sufficient visual features (e.g., buildings, objects) for tracking.
- The smartphone runs Android 10+ or iOS 14+ with support for native OpenCV integration.
- Users grant camera and sensor permissions at startup.
- Testing environments will include both controlled laboratory tests and real-world urban conditions.
- External dependencies include:
 - OpenCV Native Library
 - Platform SDKs (Android Studio / Xcode)

3. Functional requirements

3.1. Core Functionalities

3.1.1. Camera-Based Motion Tracking

- The system shall continuously capture video frames from the device's rear camera.
- The system shall detect and track visual features between consecutive frames using visual odometry algorithms (e.g., ORB or FAST).
- The system shall estimate the relative displacement (translation and rotation) based on feature correspondence.

3.1.2 Inertial Data Integration (Dead Reckoning)

- The system shall access and read data from the smartphone's accelerometer and gyroscope sensors.

- The system shall compute motion estimates (velocity and orientation) when visual tracking is unavailable or degraded.
- The system shall fuse the visual odometry and inertial data to maintain continuous position estimation.

3.1.3 Sensor Fusion Logic

- The system shall combine visual and inertial data using a weighted model or complementary filter to minimize drift and noise.
- The fusion logic shall ensure real-time updates at a minimum frequency of 30 Hz.

3.1.4 Path Visualization and UI Interaction

- The system shall render a 2D path visualization on the user's screen representing movement over time.
- The interface shall update dynamically as new position estimates are computed.
- The system shall allow the user to start, pause, and stop tracking sessions.
- The interface shall display key session metrics: distance traveled, duration, and estimated accuracy.

3.1.5 Offline Operation

- The system shall function without internet connectivity or GNSS access.
- All computations (camera, sensor, and logic) shall be performed locally on the device.

3.1.6 System Initialization and Calibration

- Upon startup, the app shall request permission to access the camera and motion sensors.
- The user shall be prompted to hold the device still for a few seconds during calibration.
- Calibration data shall be used to establish an initial reference frame.

3.1.7 Error Handling and Feedback

- The system shall detect and notify the user when visual data is unreliable (e.g., poor lighting, motion blur).

- The system shall handle camera interruptions gracefully (e.g., when the app is minimized).
- The user shall receive simple feedback messages such as “Low Light Detected,” “Tracking Lost,” or “Recalibrating.”

3.1.8 Session Management

- The system shall allow users to start new navigation sessions.
- Each session shall maintain its own data for evaluation (stored locally, temporarily).
- The system shall allow the user to view a summary of their path upon session completion.

4. Non-Functional requirements

4.1. Performance

- The system shall maintain a processing latency of less than 200 milliseconds per frame.
- The app shall maintain at least 30 FPS during real-time tracking on mid-range smartphones.
- The fusion algorithm shall process data in real time using multi-threading and optimized native modules (C++).

4.2. Reliability

- The system shall recover from short-term data loss (e.g., camera occlusion, sudden motion).
- The accumulated drift shall not exceed 10% error over 200 meters under typical urban conditions.
- The app shall be able to resume tracking after a temporary camera freeze or background state.

4.3. Compatibility

- The app shall run on Android smartphones equipped with a standard camera and IMU sensors.
- The system shall support both ARM64 and x86 architectures.

4.4. Usability

- The interface shall be minimal, intuitive, and readable in outdoor conditions.
- The app shall provide color-coded or text-based feedback on tracking status.

4.5. Maintainability

- The app architecture shall separate the UI (Java/Kotlin) from native processing modules.
- Code shall be modular and documented.
- Version control shall be maintained using Git.

4.6. Security and Privacy

- All visual and sensor data shall be processed locally on the user's device.
- Only processed metadata (e.g., path summaries and performance logs) may be stored securely.
- The app shall not upload or share any camera or sensor data externally.
- All communication with any backend shall be encrypted using HTTPS.

5. System flows

5.1. Diagram Reference

(The following description corresponds to the flow diagram provided in the project)

- The left vertical lane describes the **main operational flow** from app launch to session summary.
- The right panel contains **system states** and **exception handling branches**.
- Exception flows connect directly from the real-time processing pipeline (visual tracking + IMU integration), ensuring immediate fallback behavior.

Main Flow Description

1. User opens the application.
2. The app requests required permissions (camera and motion sensors).
3. Calibration process begins — the user is prompted to hold the device steady for 3–5 seconds.
4. Live video capture starts using the phone's rear camera.
5. Visual features are detected and tracked frame-to-frame using Visual Odometry.
6. IMU data is read continuously from the accelerometer and gyroscope.
7. The sensor fusion module combines IMU and visual data into a unified motion estimate.
8. Real-time position and orientation are calculated.
9. Path visualization is updated on the UI continuously.
10. The user may pause, resume, or stop tracking at any time.
11. Upon stopping, a session summary is displayed (distance, duration, accuracy, etc.).

System States

The system transitions through the following operational states:

- **Idle** – Application launched, permissions pending.
- **Calibrating** – Device alignment and sensor baseline initialization.
- **Tracking** – Full VO + IMU fused motion tracking.
- **Paused** – System temporarily suspended by the user.
- **Error / Recovery** – Temporary loss of visuals or sensors; recovery attempts.
- **Completed** – Tracking ended, summary displayed.

Exception Flows

Camera Lost :

- Trigger: Camera feed interrupted.
- System Action: Tracking pauses, system retries automatically.

Low Light Detected :

- Trigger: Poor visibility / insufficient features.
- System Action: Switch to **Dead Reckoning (IMU-only)** mode until visuals recover.

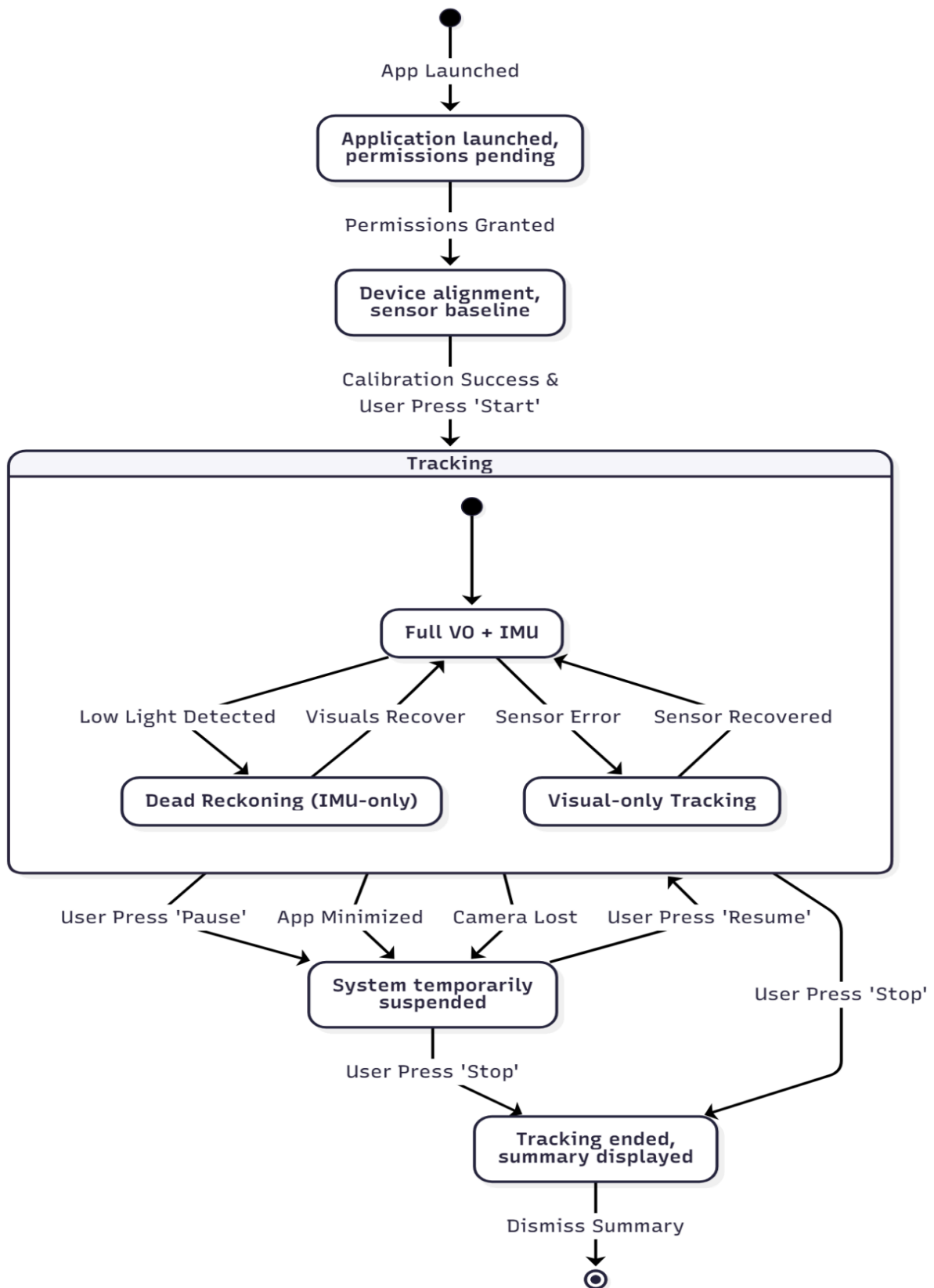
Sensor Error :

- Trigger: IMU malfunction or noisy readings.
- System Action: Temporarily rely on **visual-only** tracking.

App Minimized / Background State :

- Trigger: User leaves the app or the screen turns off.
- System Action: Tracking pauses to save battery and prevent corrupted readings.

State diagram :



6. Risk management

6.1. The main risks identified for the NavSight project include:

- **Sensor inaccuracies:** IMU drift or calibration errors may lead to accumulated positional error.
Mitigation: Implement sensor fusion filters and periodic recalibration prompts.
- **Lighting and visual limitations:** Poor lighting or motion blur can degrade visual odometry accuracy.
Mitigation: Use adaptive exposure control and switch to Dead Reckoning mode.
- **Performance bottlenecks:** Real-time processing may overload the device's CPU or GPU.
Mitigation: Optimize native C++ algorithms and limit frame rate during low activity.
- **User permissions denial:** Users may reject camera or sensor permissions.
Mitigation: Provide clear explanations and retry logic.
- **Battery drain:** Continuous sensor and camera use may increase power consumption.
Mitigation: Implement energy-efficient mode and suspend tracking in the background.

7. System main screen specifications

7.1. The main screen of NavSight will consist of:

- **Live Camera View:** displays real-time visual feed with overlay of detected features.
- **Path Visualization Layer:** shows the user trajectory as a dynamic line on the screen.
- **Status Panel:** indicates calibration, tracking accuracy, and current mode (VO/DR).
- **Control Buttons:** Start / Pause / Resume / Stop tracking.
- **Session Summary Button:** opens the summary screen after session completion.

8. Non-goals

8.1. The following aspects are explicitly excluded from the project scope:

- Cloud-based storage or remote data transmission.
- Integration with external GNSS or mapping APIs.
- 3D reconstruction or SLAM mapping.
- Cross-platform (iOS) support.
- Voice guidance or audio-based navigation

9. Open issues

9.1. Camera and IMU Synchronization:

Further optimization is needed to ensure precise synchronization between camera frames and IMU sensor readings. Minor desynchronization currently causes a slight drift in position estimation.

9.2. Lighting Adaptation:

Additional logic must be implemented to handle extreme lighting changes, such as transitions between bright sunlight and tunnels, which affect feature detection accuracy.

9.3. Device Compatibility:

Testing has so far been limited to several Android devices. Broader testing across different manufacturers and hardware capabilities is required.

9.4. Drift Compensation:

A lightweight correction mechanism (loop detection or periodic re-centering) needs to be finalized to prevent long-term path drift.

9.5. User Interface Design:

The final layout for displaying the path, sensor data, and tracking status is still under review to ensure a clean, rider-safe interface.

9.6. Performance and Energy Profiling:

Additional field testing is required to evaluate battery impact and optimize CPU/GPU load during continuous operation.

10. References

- 10.1. Scaramuzza, D., & Fraundorfer, F. (2011). *Visual Odometry: Part I - The First 30 Years and Fundamentals*. IEEE Robotics & Automation Magazine.
- 10.2. Mur-Artal, R., Montiel, J., & Tardos, J. D. (2015). *ORB-SLAM: A Versatile and Accurate Monocular SLAM System*. IEEE Transactions on Robotics.
- 10.3. Engel, J., Schöps, T., & Cremers, D. (2014). *LSD-SLAM: Large-Scale Direct Monocular SLAM*. ECCV.
- 10.4. Zhou, G., & Liu, Y. (2020). *Visual-Inertial Navigation Systems: A Review*. Sensors.
- 10.5. Google VPS Overview - <https://developers.google.com/maps/documentation/visual-positioning>
- 10.6. OpenCV Documentation - <https://docs.opencv.org/>
- 10.7. Kotlin Documentation - <https://kotlinlang.org/>
- 10.8. OpenCV Integration with android - <https://opencv.org/android/>
- 10.9. RPG Zurich - *SLAM Research Overview* - <https://rpg.ifi.uzh.ch>